

Componentele unui sistem de calcul

Unitatea de învățare nr. 4 · Programarea calculatoarelor și limbaje de programare I

Obiective și cuprins

Obiectivele unității

- Descrierea **principalelor componente** ale unui sistem de calcul și a **relațiilor** dintre ele
- Descrierea **caracteristicilor sistemului de memorii**
- Descrierea principalelor **tehnici de gestiune a memoriei** folosite de sistemele de operare
- Timp de studiu: **2–3 ore**

1. Concepte fundamentale privind arhitectura calculatoarelor
2. Componentele unui sistem de calcul
3. Caracteristicile sistemului de memorii; metode de acces la date
4. Ierarhia de memorii
5. Adresarea memoriei

+ Test de autoevaluare rezolvat pas cu pas · Rezumat

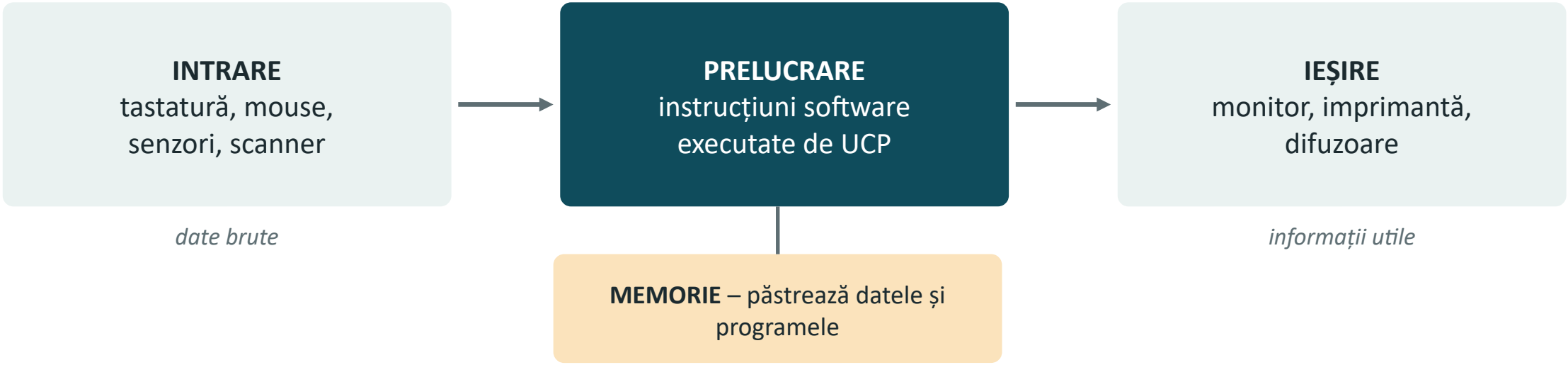
1

Concepte fundamentale privind arhitectura calculatoarelor

Ce este un calculator, din ce puncte de vedere îl descriem și cum se organizează ierarhic (cele 6 nivele Tanenbaum)

Ce este un calculator?

Un calculator = combinație de componente **hardware** și **software** prin care **datele** colectate de la dispozitivele de intrare sunt convertite în **informații utile** prin executarea unor instrucțiuni software; rezultatul este trimis către dispozitivele de ieșire.



Observație: aceeași mașină poate face sarcini complet diferite doar prin schimbarea **programului** (software-ului) – hardware-ul rămâne același.

Trei puncte de vedere asupra unui sistem de calcul

A Arhitectural

- Caracteristicile care interesează **programatorul** – rol în execuția logică a programului
- Setul de **operații** suportate
- **Tipuri de date** și numărul de biți al reprezentării
- Mecanismele de **I/E**, **regiștrii**, mecanismele de **adresare** a memoriei

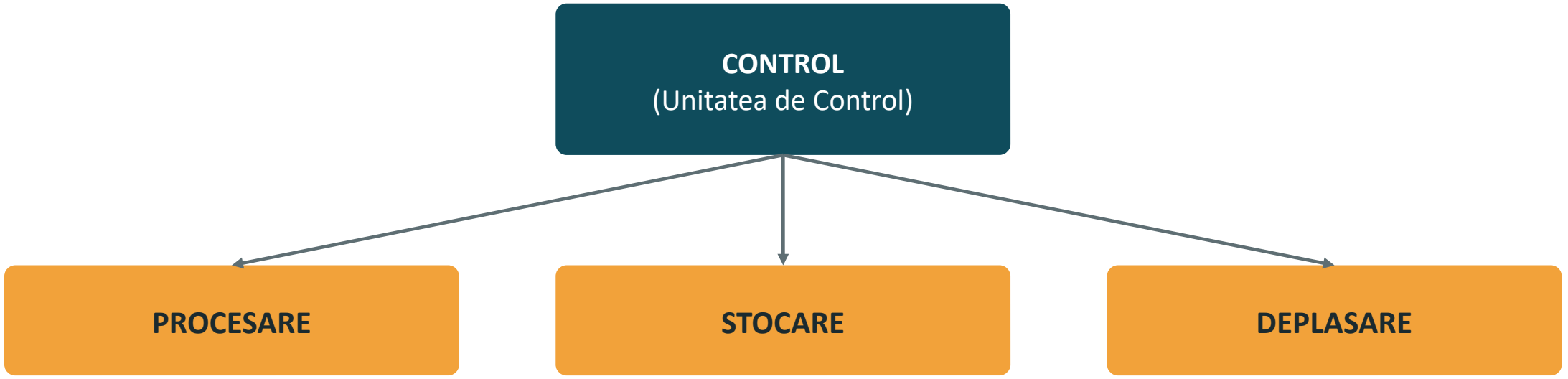
S Structural

- **Unitățile operaționale** și legăturile dintre ele
- **Cum** este implementată arhitectura
- Semnalele de control ale unității de control
- Tehnologia memoriilor, interfețele cu perifericele, circuite logice combinaționale/secvențiale

F Funcțional

- **Ce face** calculatorul – 4 funcții de bază:
- **procesarea** datelor
- **stocarea** datelor în memorie
- **deplasarea** datelor (memorie ↔ periferice, între locații)
- **controlul** acestor activități (Unitatea de Control)

Punctul de vedere funcțional: cele 4 funcții de bază



datele, aflate în diferite formate, sunt prelucrate

datele sunt păstrate în memorie pentru accesări ulterioare

memorie → ieșire, intrare → memorie, între locații de memorie

Observație: controlul coordonează celelalte trei funcții: decide **ce** se prelucrează, **unde** se stochează și **încotro** se deplasează datele.

Organizarea ierarhică: cele 6 nivele (Tanenbaum)

5	**Limbaj orientat pe problemă**	C, C++, Java – majoritatea programatorilor de aplicații
4	**Limbaj de asamblare**	formă simbolică a codului pentru nivelele inferioare
3	**Sistem de operare**	biblioteci, alocare memorie, întreruperi, I/E
2	**Set de instrucțiuni (ISA)**	primul nivel programabil; interfața software ↔ hardware
1	**Microarhitectură**	microinstrucțiuni: semnale de control pentru calea de date
0	**Nivelul logic**	circuite digitale; operații de logică booleană

abstractizare ↑ / execuție ↓

Ideea

- Fiecare nivel = o **mașină** (fizică sau virtuală) cu propriul limbaj
- Fiecare nivel e construit **pe baza celui inferior**
- Doar programele de la nivelul **0** rulează direct pe circuite
- Celelalte sunt **traduse** sau **interpretate** spre un nivel inferior

Nivelele 0–2: de la circuite la setul de instrucțiuni

Nivelul 0 – logic

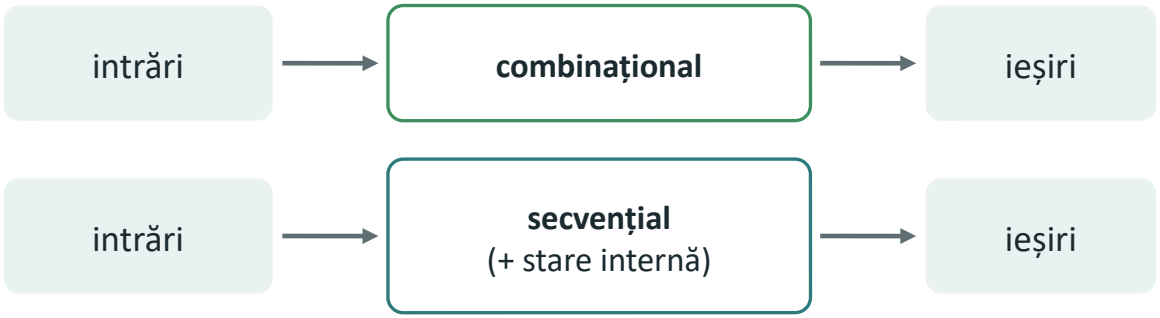
- Circuite digitale; operațiile de bază = **logică booleană**
- **Secvențiale**: ieșirile depind de intrări **și** de ordinea aplicării lor (au memorie) – ex. **contoare, regiștri de deplasare**
- **Combinaționale**: ieșirile depind **doar** de intrările prezente; **nu pot stoca** informații – ex. **sumatoare, multiplicatoare, comparatoare**

Nivelul 1 – microarhitectură

Microinstrucțiuni: setul de semnale de control pentru **calea de date** (UAL + intrările/ieșirile ei), pentru un ciclu de execuție.

Nivelul 2 – setul de instrucțiuni

Primul nivel programabil de utilizator: ce **poate face** hardware-ul – instrucțiuni mașină, formatul lor, tipuri de date, moduri de adresare, regiștri, structura memoriei.



Observație: „memoria” unui circuit secvențial = starea internă care ține minte ce s-a întâmplat înainte; un circuit combinațional „uită” imediat.

Nivelele 3–5: sistem de operare, asamblare, limbaje de nivel înalt

Nivelul 3 – sistemul de operare

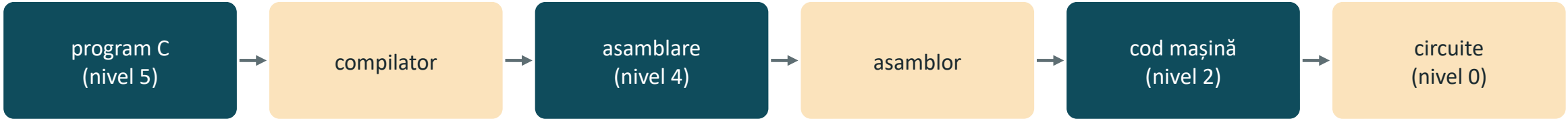
- Adaugă **instrucțiuni noi** față de nivelul 2
- Limbaj special pentru **resurse partajate**
- **Biblioteci** (ex. alocarea memoriei)
- Tratează **întreruperile** și operațiile de **I/E**

Nivelul 4 – limbaj de asamblare

- Limbaj **foarte detaliat**, simbolic
- Avantaj: **viteză** și acces la caracteristicile hardware
- Ex.: multe **compilatoare** sunt scrise în asamblare

Nivelul 5 – orientat pe problemă

- Limbaje de **nivel înalt**: C, C++, Java
- Aici lucrează **majoritatea** programatorilor
- Un algoritm → programe diferite, în limbaje diferite



*Programatorul de aplicații nu e interesat **cum** se execută instrucțiunile (direct, interpretat sau tradus).*

2

Componentele unui sistem de calcul

Hardware, software, date, comunicare · UCP (ALU, Unitatea de Control, regiștri) · frecvență, MIPS, CPI · Legea lui Amdahl

Cele 4 componente ale unui sistem de calcul

H Hardware

Mecanismele **fizice** pentru intrări (input), rezultate (output), **procesare** și controlul componentelor de I/E și de stocare.

S Software

Programe de aplicație (sarcini specifice unui utilizator) și **programe de sistem** (sistem de operare, compilatoare, linkeditoare, asambloare).

D Date

Informațiile manipulate și prelucrate: **numerice, alfanumerice, grafice**… reprezentate într-o formă utilizabilă de calculator.

C Comunicare

Hardware + software care asigură **transportul** programelor și datelor între sisteme informatice **interconectate**.

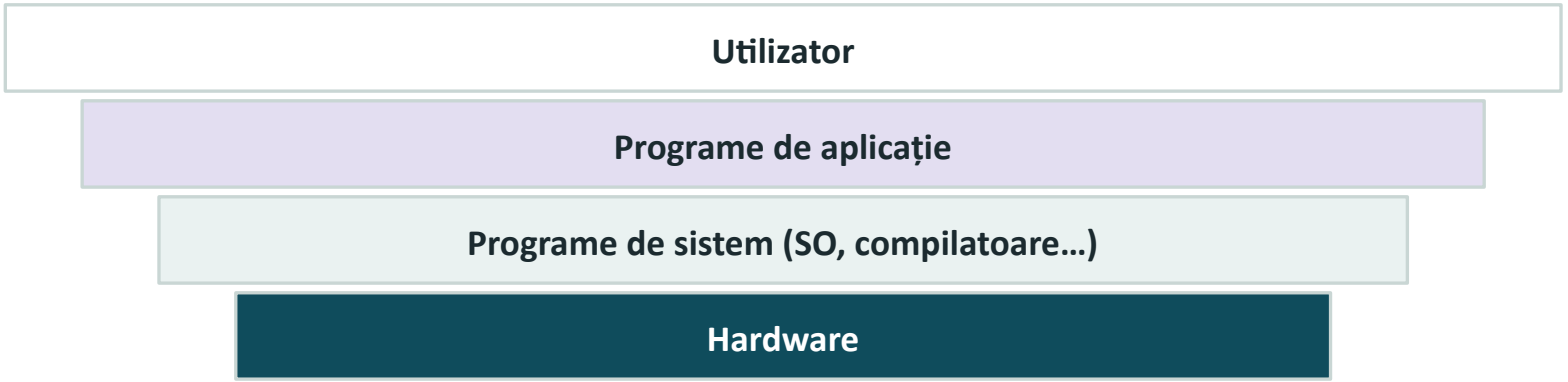
Software: programe de aplicație vs. programe de sistem

Programe de aplicație

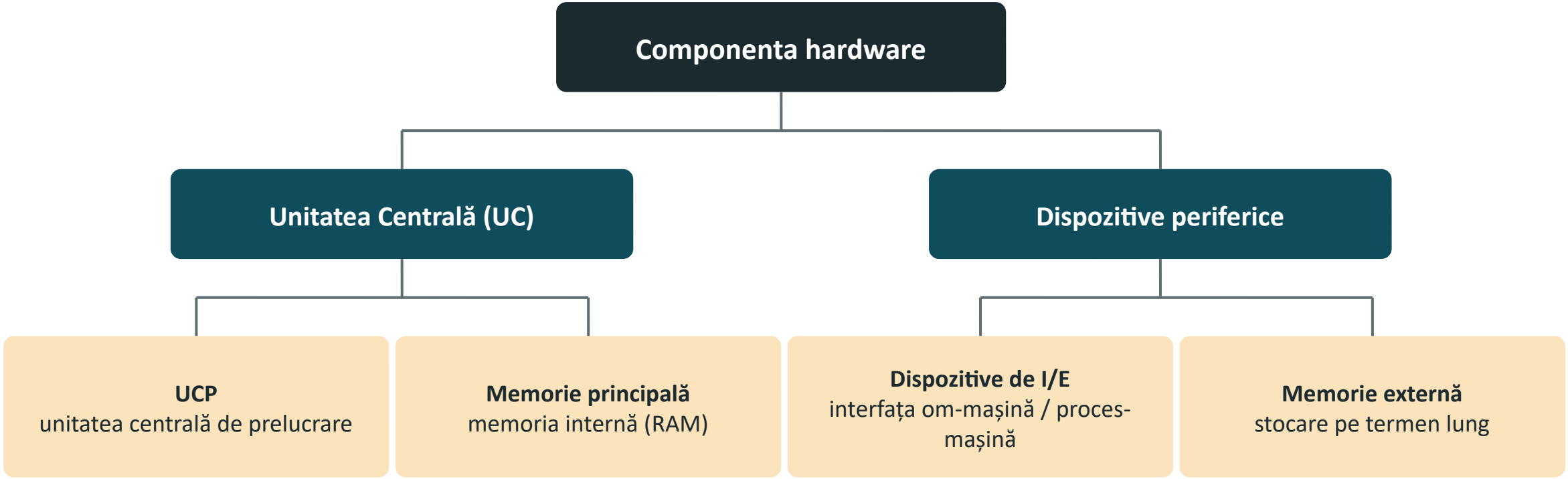
- Execută activități cu **caracter particular**, specifice unui utilizator
- = programele utilizatorilor
- Ex.: editor de text, joc, program de calcul tabelar, programul vostru în C

Programe de sistem

- Execută activități **comune** tuturor sistemelor de calcul
- **Sistemul de operare**, **compilatoare**, **linkeditoare**, **asambloare** etc.



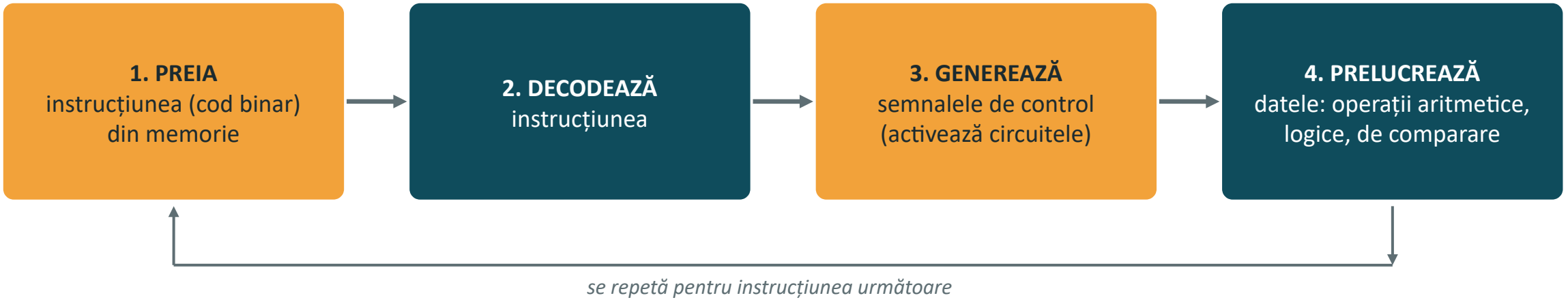
Componenta hardware – schema de bază (Fig. 1)



Observație: **Memoria internă (centrală)** păstrează temporar instrucțiunile și datele programului (șiruri de biți) în timpul execuției și interacționează direct cu UCP. Este **RAM** (acces aleator): orice locație e găsită „instantaneu”, **același timp de acces** indiferent de locație.

UCP – rolul unității centrale de prelucrare

La PC-uri, UCP = **microprocesorul**. Înainte de execuție, instrucțiunile și datele trebuie să fie **în memorie**; apoi fiecare instrucțiune este adusă pe rând în UCP.



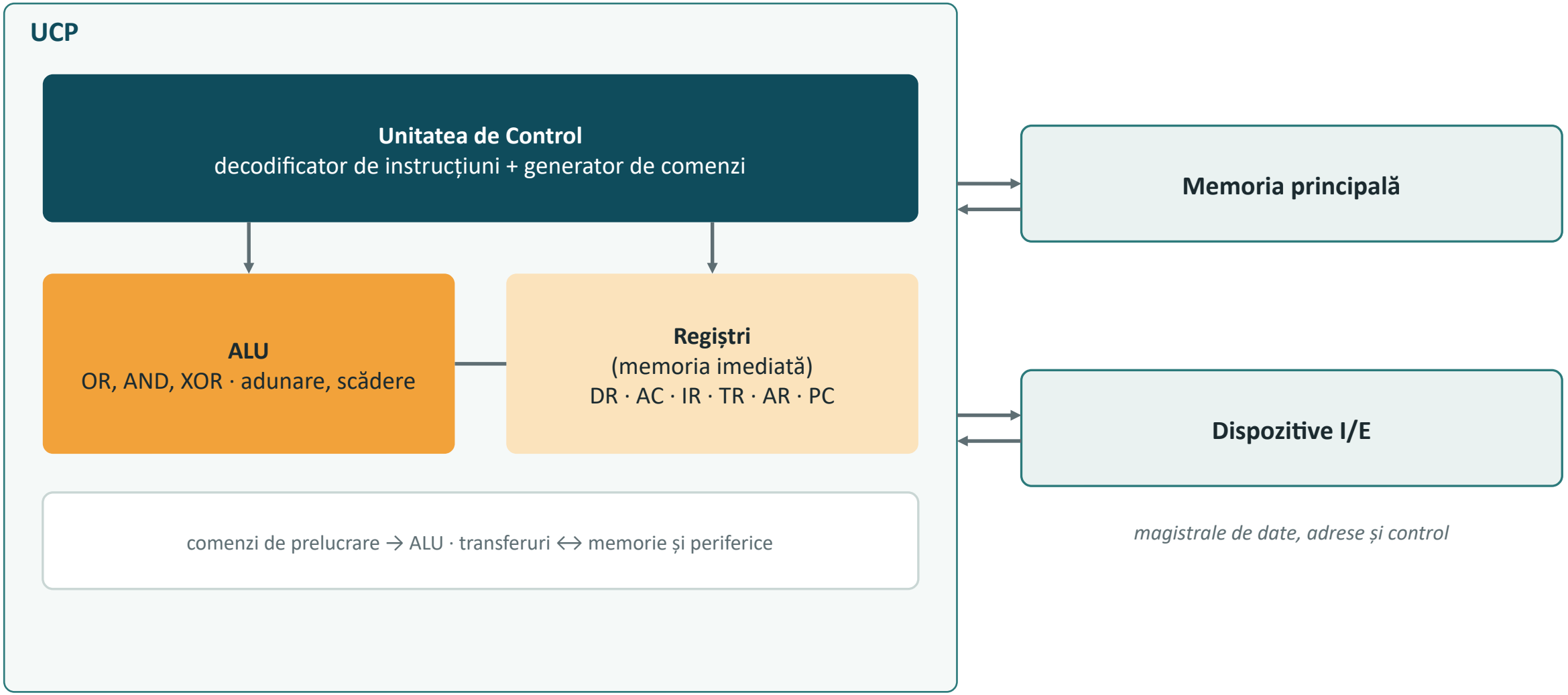
UCP mai face

Dirijează celelalte dispozitive și controlează aproape toate sarcinile importante ale calculatorului.

Trei componente principale

ALU (unitatea aritmetico-logică) · **Unitatea de Control** · **Memoria imediată** (regiștrii)

Structura internă a UCP



ALU și Unitatea de Control

ALU – Unitatea Aritmetico-Logică

- Operații **logice**: OR, AND, exclusive OR (XOR)
- Operații **aritmetice**: adunare, scădere – pe secvențe binare
- Gestionează **toate cererile de calcul** (de la utilizatori și de la alte componente)

Unitatea de Control – 3 sarcini

- **Monitorizează** hardware-ul ca să fie activate comenzile programului curent
- **Controlează intrarea și ieșirea** datelor: semnalul potrivit, la locul potrivit, la momentul potrivit
- **Controlează fluxul de date** în UCP

Exemplu ALU pe 4 biți (ilustrativ):

A	0	1	1	0
B	0	0	1	1
A AND B	0	0	1	0
A OR B	0	1	1	1
A XOR B	0	1	0	1
A + B	1	0	0	1

A = 6, B = 3 → A + B = 9 = 1001_2

Regiștrii procesorului (memoria imediată)

Regiștrii = celule de memorie din UCP pentru stocarea **temporară** a datelor prelucrate și a **adreselor**. **Dimensiunea regiștrilor determină dimensiunea datelor** cu care lucrează procesorul.

Registru	Nume	Rol	Biți
DR	Data Register	reține **operandul** citit din memorie	16
AC	Accumulator	registru de prelucrare cu **scop general**	16
IR	Instruction Register	**instrucțiunea** citită din memorie	16
TR	Temporary Register	stocare **temporară** în timpul procesării	16
AR	Address Register	**adresa** de memorie (lățimea unei adrese)	12
PC	Program Counter	adresa **următoarei instrucțiuni** de citit	12

Dimensiunile 16/12 biți sunt cele din exemplul de calculator simplu din curs; AC, IR, TR – presupuse de aceeași lățime cu DR.

DR pe 16 biți

2^{16} = **65 536** combinații binare posibile

AR / PC pe 12 biți

2^{12} = **4 096** adrese distincte de memorie

Exemplu pas cu pas: UCP împarte două numere

Scenariul din curs: Unitatea de Control încarcă două numere din memorie în regiștri și cere ALU să le **împartă** (aritmetic) sau să decidă dacă sunt **egale** (logic).

1. **PC** conține adresa instrucțiunii; ea se copiază în **AR**, iar instrucțiunea citită ajunge în **IR**
2. **Unitatea de Control** decodifică instrucțiunea din IR (decodificatorul de instrucțiuni)
3. Primul operand (ex. 84) este citit din memorie în **DR** și mutat în **AC**
4. Al doilea operand (ex. 12) este citit în **DR**
5. UC trimite comanda „împarte”: **ALU** calculează 84 / 12 = 7, rezultatul ajunge în **AC**
6. **PC** avansează la adresa următoarei instrucțiuni → ciclul se reia

Starea finală (ilustrativ)

PC	n+1
IR	DIV
DR	12
AC	7

Valorile numerice sunt ilustrative.

Frecvența de ceas

Operațiile elementare (preluare instrucțiune, decodificare, încărcare date, operații aritmetice/logice) sunt reglate de **ceasul sistemului** – un circuit care generează un flux de impulsuri.



Perioada de ceas

= **cel mai mic element de timp** al procesorului
$t = 1 / f$

Unități

- Frecvența = cicluri pe secundă: **MHz** (10^6) sau **GHz** (10^9)
- O instrucțiune necesită de regulă **mai multe cicluri** (câteva sau zeci)
- Procesoarele moderne execută **mai multe instrucțiuni pe ciclu** (preluare paralelă)

Exemplu (ilustrativ)

$f = 3\ GHz = 3 \cdot 10^9$ cicluri/s
$t = 1 / (3 \cdot 10^9)\ s \approx$ **0.33 ns** per ciclu
Într-o secundă: 3 miliarde de cicluri.

Viteza de lucru: MIPS, CPI și timpul de execuție

$t_{UCP} = 1 / f$	timpul unei operații elementare (un ciclu); f = frecvența
IC	Instruction Count – nr. de instrucțiuni mașină executate de program
CPI_i , I_i	cicluri pentru o instrucțiune de tip i · câte instrucțiuni de tip i se execută
$CPI = \Sigma (CPI_i \cdot I_i) / IC$	numărul **mediu** de cicluri de ceas pe instrucțiune
$T = IC \cdot CPI \cdot t_{UCP}$	timpul de execuție al programului
$MIPS = IC / (T \cdot 10^6) = f / (CPI \cdot 10^6)$	milioane de instrucțiuni pe secundă

Atenție: MIPS depinde de **setul de instrucțiuni** al procesorului și de complexitatea programelor → comparația între procesoare cu seturi diferite de instrucțiuni este **neconcludentă**.

Exemplu ilustrativ pas cu pas: CPI, T și MIPS

Procesor cu **f = 2 GHz**; program cu **IC = 1 000 000** instrucțiuni, de trei tipuri:

Tip instrucțiune	I_i	CPI_i	$CPI_i \cdot I_i$
aritmetico-logică	500 000	1	500 000
acces la memorie	300 000	2	600 000
salt	200 000	3	600 000
Total	IC = 1 000 000		**1 700 000 cicluri**

Observație: cu aceeași frecvență, un **CPI mai mic** înseamnă un procesor mai rapid; datele numerice sunt alese pentru exemplificare (nu apar în curs).

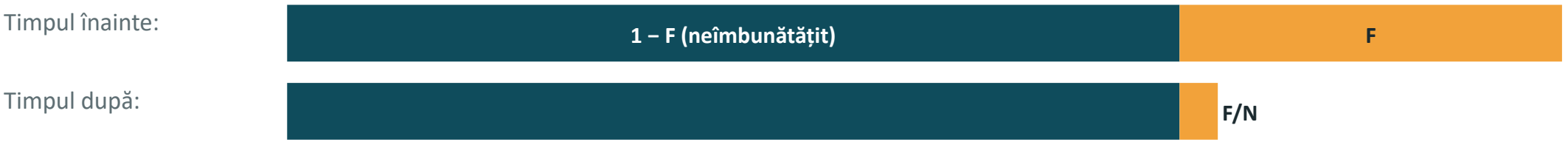
1. CPI = 1 700 000 / 1 000 000 = **1.7** cicluri/instrucțiune
2. $t_{UCP} = 1 / (2 \cdot 10^9) =$ **0.5 ns**
3. $T = IC \cdot CPI \cdot t_{UCP} = 10^6 \cdot 1.7 \cdot 0.5$ ns = **0.85 ms**
4. $MIPS = f / (CPI \cdot 10^6) = 2 \cdot 10^9 / (1.7 \cdot 10^6) \approx$ **1176 MIPS**
5. Verificare: $IC / (T \cdot 10^6) = 10^6 / (0.85 \cdot 10^{-3} \cdot 10^6) \approx 1176$ ✓

Legea lui Amdahl – câștigul de performanță (speedup)

O îmbunătățire într-o singură direcție (procesoare paralele, cache, frecvență dublă…) **nu** implică o îmbunătățire corespunzătoare a performanței de ansamblu.

Varianta inițială (paralelizare pe N procesoare)
T = timpul pe un procesor; F = fracțiunea paralelizabilă
speedup = T / [T(1−F) + T·F/N]
= **1 / [(1 − F) + F/N]**

Varianta generalizată (o componentă de N ori mai bună)
speedup = perf. nouă / perf. veche
= timp vechi / timp nou
= **1 / [(1 − F) + F/N]**, F = fracțiunea de timp în care e folosită componenta

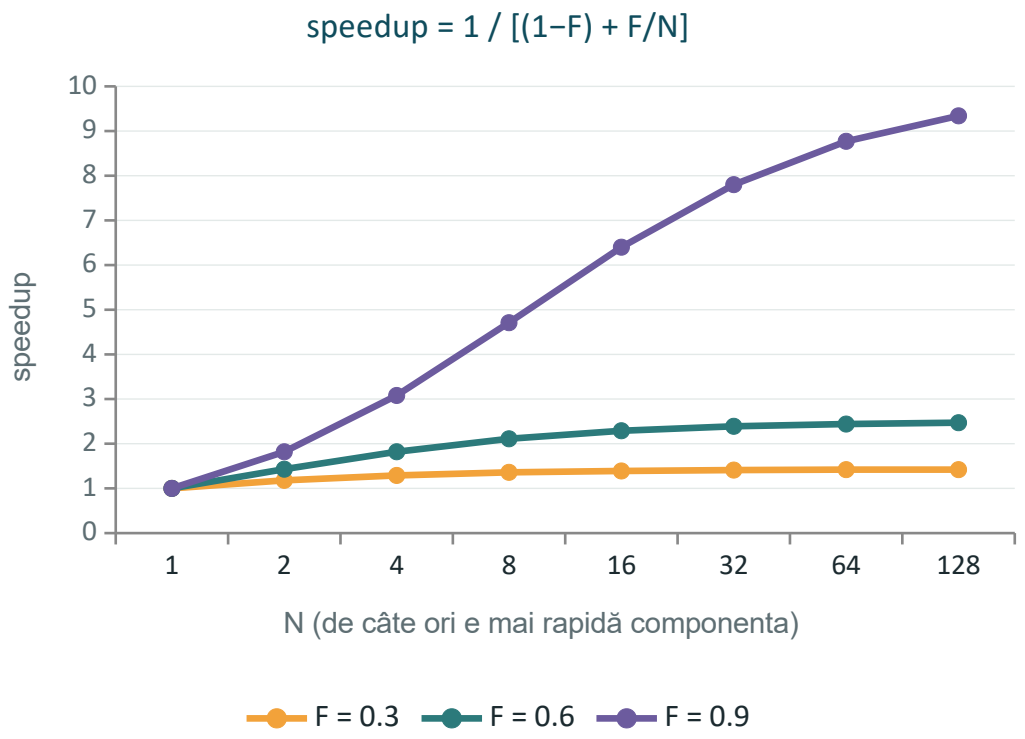


Observație: partea **neîmbunătățită** (1 − F) rămâne neschimbată și ajunge să domine timpul total.

Legea lui Amdahl – exemplul din curs, pas cu pas

O îmbunătățire face rularea de **10 ori mai rapidă** (N = 10), dar este folosită doar **30%** din timp (F = 0.3).

1. Partea neîmbunătățită: 1 − F = 1 − 0.3 = **0.7**
2. Partea îmbunătățită, accelerată: F / N = 0.3 / 10 = **0.03**
3. Timpul nou (relativ): 0.7 + 0.03 = **0.73**
4. speedup = 1 / 0.73 ≈ **1.37** (doar 37% mai rapid, deși componenta e de 10× mai rapidă!)
5. Limita pentru N → ∞: F/N → 0, deci speedup → 1 / (1 − F) = 1 / 0.7 ≈ **1.43**



Concluzie: când **F e mic**, viteza de ansamblu **nu crește semnificativ**; plafonul este 1/(1−F): 1.43 (F=0.3), 2.5 (F=0.6), 10 (F=0.9).

3

Caracteristicile sistemului de memorii; metode de acces la date

Amplasare, metode de acces (secvențial, direct, aleator, asociativ), organizare, capacitate, performanță

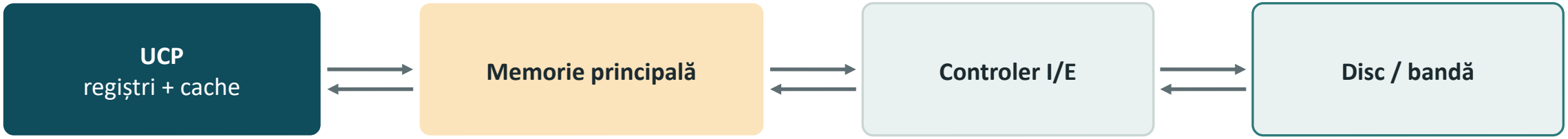
Amplasarea memoriei: internă vs. externă

Memoria internă

- În **interiorul** calculatorului
- Considerată **memoria principală**
- Alte forme interne: **memoria cache** și **regiștrii** (stocarea temporară a datelor prelucrate de ALU)

Memoria externă

- **Dispozitive de stocare periferice**: discuri, benzi magnetice
- Accesibile procesorului **prin controlere (module) de I/E**



Metode de acces la date (1): secvențial și direct

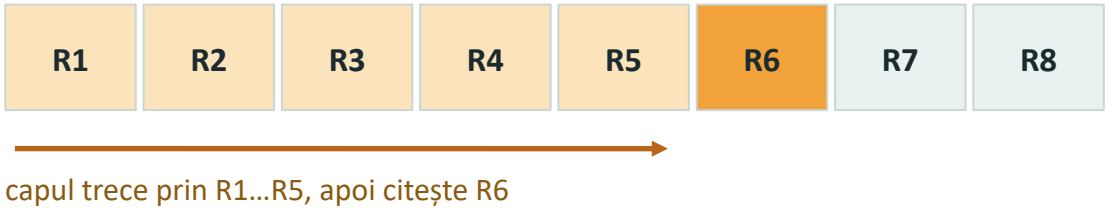
Acces secvențial

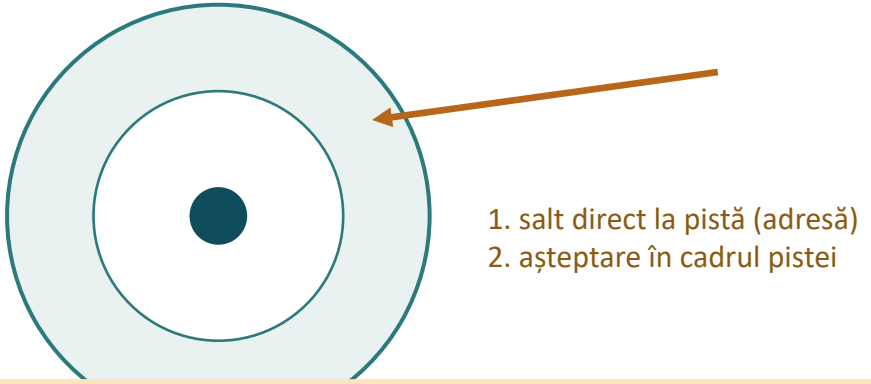
- Date organizate în **înregistrări**, accesate într-o **secvență liniară**
- Un mecanism de citire-scriere **partajat**, care trece peste fiecare înregistrare intermediară
- Pentru a n-a înregistrare → se parcurg primele **n – 1**
- Timp de acces **variabil**. Ex.: **banda magnetică**

Acces direct

- Tot un mecanism de citire-scriere **partajat**
- Blocurile au o **adresă unică** bazată pe locația fizică
- Timp **variabil** – depinde de poziția înregistrării accesate **anterior**
- Ex.: **unitățile de disc fix (hard-disk)**

Acces la înregistrarea 6:





Observație: secvențial = parcurgi tot ce e înainte; direct = sari aproape de țintă, apoi cauți local.

Metode de acces la date (2): aleator și asociativ

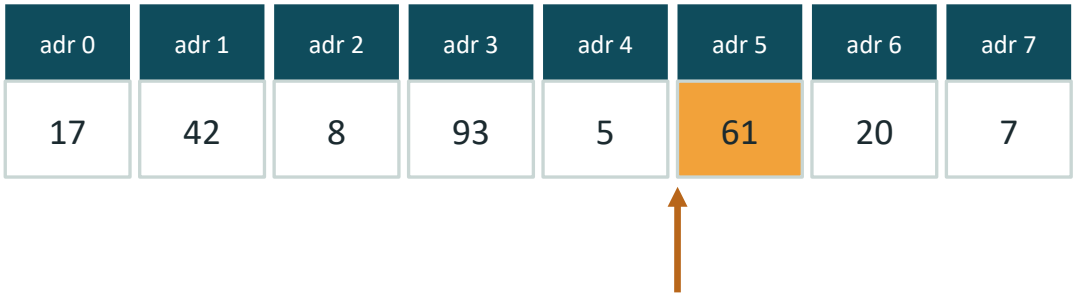
Acces aleator (RAM)

- Fiecare locație adresabilă are un **mecanism de adresare încorporat**
- Timp de acces **constant**, independent de accesările anterioare
- Orice locație poate fi selectată aleator și accesată direct
- Ex.: **memoria principală**

Acces asociativ (după conținut)

- Datele sunt identificate **după conținut**, nu după adresă
- La citire se specifică **conținutul** cuvântului sau o **parte** a lui
- Ex.: memoriile **cache** pot utiliza acces asociativ

Citește adresa 5 → direct, în același timp ca oricare alta:



Caută conținutul „cheie = 61” → toate locațiile se compară simultan:



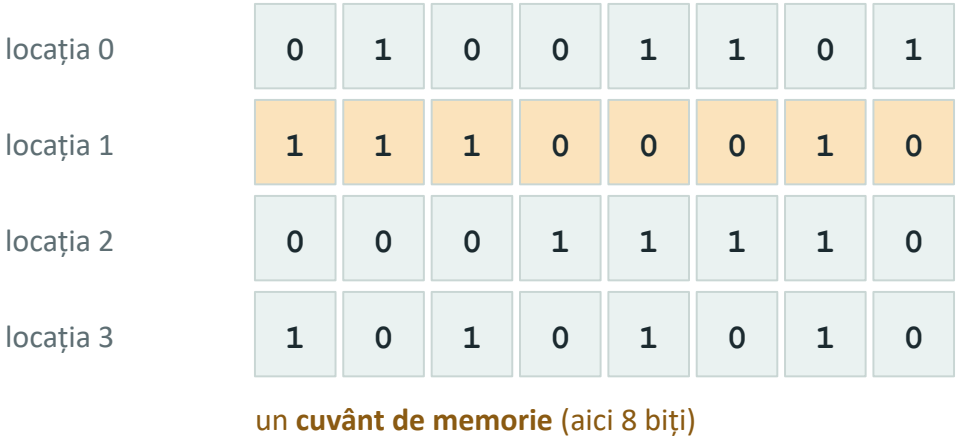
Comparație: cele 4 metode de acces

Metoda	Cum se găsește data	Timp de acces	Exemplu
Secvențial	parcurgere liniară a înregistrărilor	variabil (depinde de locație)	bandă magnetică
Direct	adresă unică pe bloc + căutare locală	variabil (depinde de poziția anterioară)	hard-disk
Aleator	adresă cu mecanism de adresare propriu	**constant**	memoria principală (RAM)
Asociativ	după conținut (sau parte din el)	rapid, comparații simultane	memoria cache

Observație: de reținut: **numai accesul aleator** are timp de acces **constant**; la secvențial și direct timpul depinde de unde se află mecanismul de citire-scriere.

Organizarea memoriei și capacitatea de stocare

Memoria = **matrice de celule**, fiecare celulă stochează **un bit**, plus circuitele pentru stocare și regăsire.



un **cuvânt de memorie** (aici 8 biți)

Termeni

- **Byte (octet)** = 8 biți – cea mai mică unitate **adresabilă**
- **Locație / cuvânt de memorie** = succesiune de biți adresată individual: **8, 16, 32 sau 64** biți

Multiplii byte-ului (fiecare treaptă × $2^{10} = 1024$)

1 KB = 2^{10} B (1 024 B)

1 MB = 2^{10} KB = 2^{20} B (1 048 576 B)

1 GB = 2^{10} MB = 2^{30} B (≈ $1.07 \cdot 10^{9}$ B)

1 TB = 2^{10} GB = 2^{40} B (≈ $1.10 \cdot 10^{12}$ B)

1 PB = 2^{10} TB = 2^{50} B (≈ $1.13 \cdot 10^{15}$ B)

Performanța memoriei: cei 3 parametri

1 Timpul de acces (latența)

RAM: timpul de la **cererea** de citire/scriere până când datele sunt **stocate/disponibile** – în **nanosecunde**. Acces direct/secvențial: timpul de **poziționare** a mecanismului de citire-scriere.

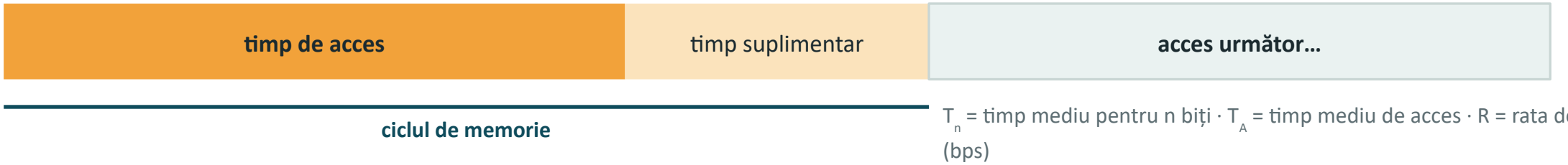
2 Ciclul de memorie

timpul de acces + timpul suplimentar necesar înainte să poată începe un al doilea acces. Are legătură cu **magistrala sistemului**, nu cu procesorul.

3 Rata de transfer

rata cu care datele intră/ies din memorie. Pentru RAM: **1 / ciclul de memorie**. Pentru celelalte: $\mathbf{T_n = T_A + n / R}$

Ciclul de memorie pe axa timpului:



T_n = timp mediu pentru n biți · T_A = timp mediu de acces · R = rata de transfer (bps)

Rata de transfer – exemplu ilustrativ pas cu pas

$T_n = T_A + n / R$ (memorii cu acces direct sau secvențial)

Date (ilustrative, nu sunt în curs): un dispozitiv cu **T_A = 5 ms**, **R = 1 Mbps = 10^6 biți/s**; vrem să citim un bloc de **1 KB = 8 192 biți**.

1. Convertim blocul în biți: 1 KB = 1 024 B · 8 = **8 192 biți** (n)
2. Timpul de transfer propriu-zis: n / R = 8 192 / 10^6 s = **8.192 ms**
3. Adunăm timpul de acces: T_n = 5 ms + 8.192 ms = **13.192 ms**



4

Ierarhia de memorii

De ce o ierarhie · localitatea referințelor · hit/miss · regiștri, RAM/ROM, cache · memoria externă și hard-disk-ul

De ce avem nevoie de mai multe tipuri de memorie?

Calculatoarele de uz general folosesc spații de stocare **suplimentare** memoriei principale, din motive **funcționale** și **economice**:

1 Spațiu insuficient

nu există destulă memorie principală pentru toate programele utilizate

2 Date acumulate

în timp se strâng volume mari de date și programe de prelucrare

3 Cost

o memorie rapidă (cât procesorul) este **foarte scumpă**

Memoria principală (internă)

Accesată **direct de UCP**. Conține **doar** programele și datele **utilizate curent**.

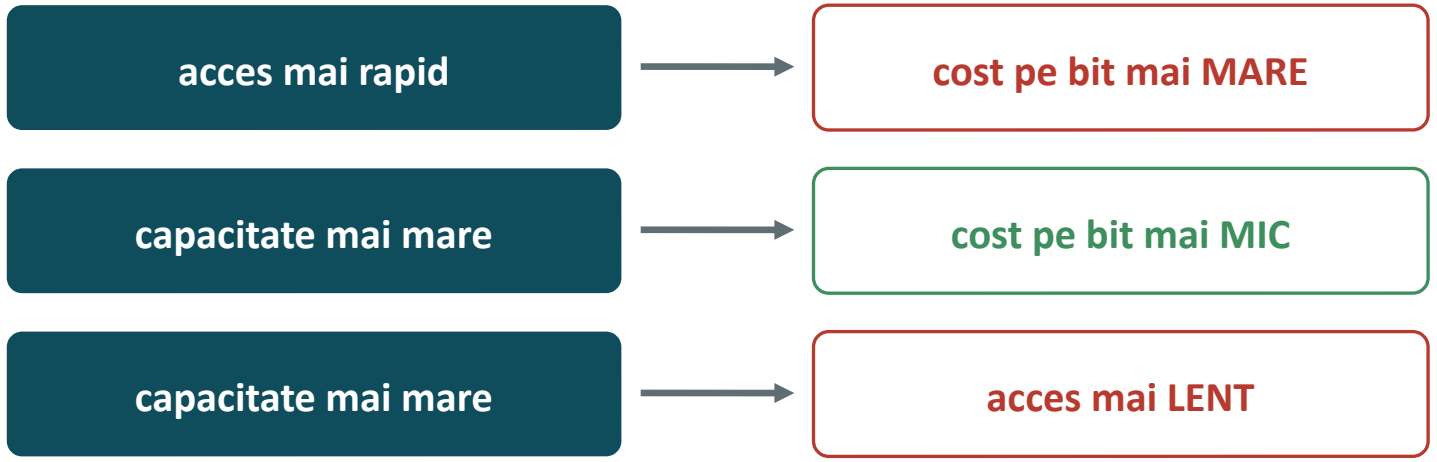
Memoria auxiliară (secundară)

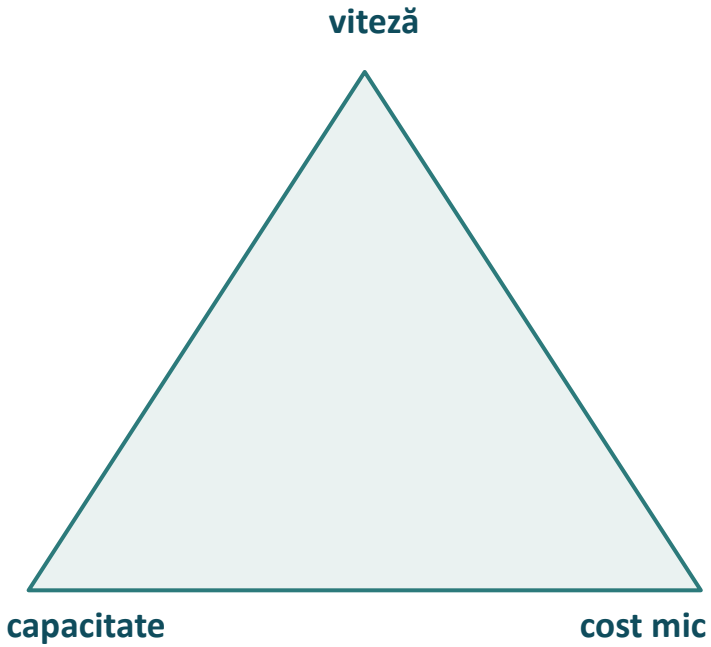
Stocare de **backup**: HD, floppy, CD-ROM, DVD-ROM… Informațiile se transferă în memoria principală **când e nevoie**.

*Dispozitivele de stocare + algoritmii de control = **sistemul de memorie** al calculatorului.*

Compromisul viteză – capacitate – cost

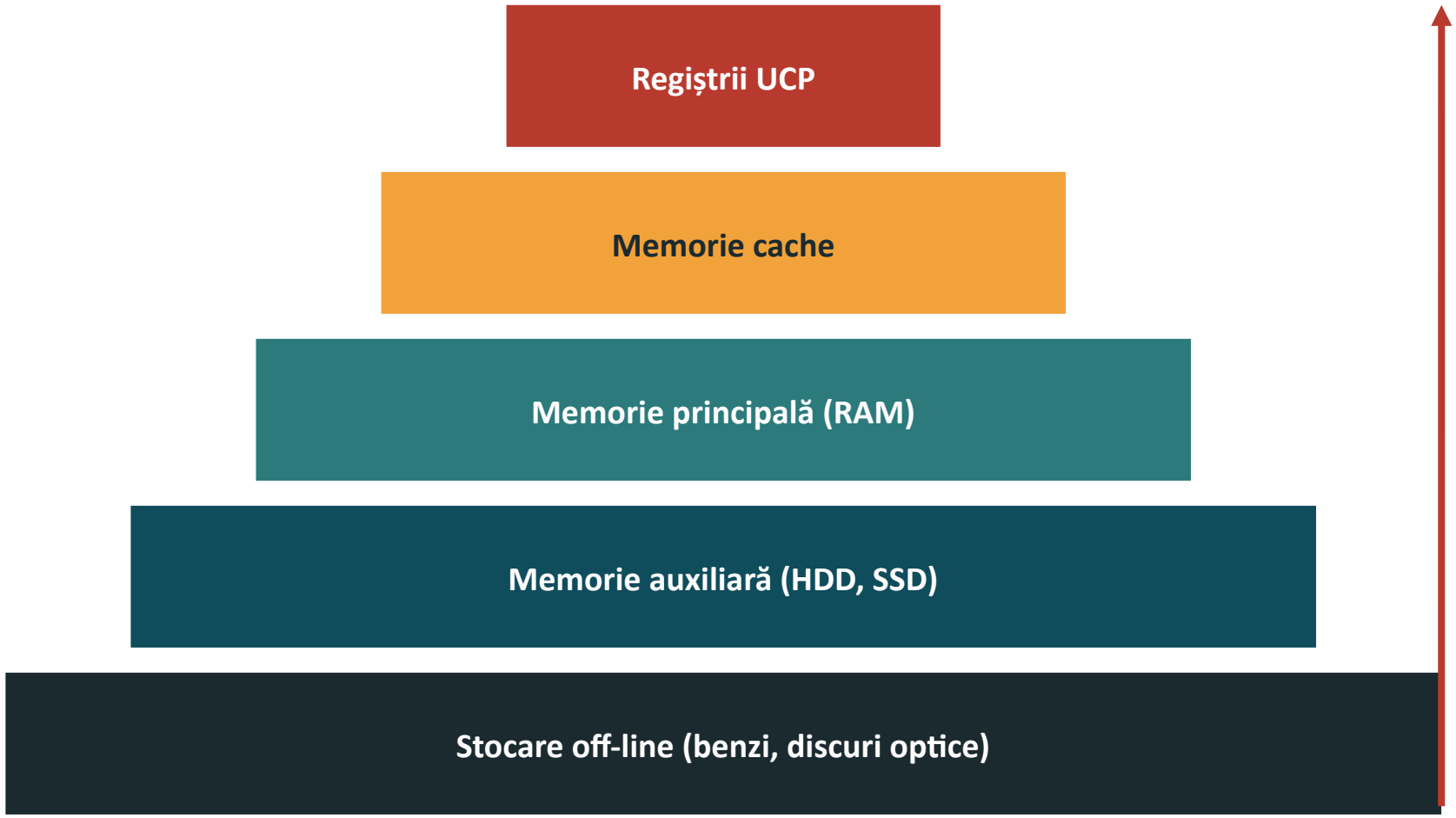
Obiective: **capacitate mare** și **acces rapid**, la un **cost** rezonabil. Dar:





Concluzie: nicio tehnologie unică nu le oferă pe toate trei → se folosesc **mai multe componente de memorie**, din tehnologii diferite, organizate într-o **ierarhie**: puțină memorie rapidă și scumpă sus, multă memorie lentă și ieftină jos.

Ierarhia de memorii (Fig. 2)



mai rapidă
mai scumpă / bit
mai mică

Nivelul de vârf e cel mai apropiat de UCP.

mai ieftină / bit
mai mare
mai lentă

Principiul localității referințelor

Observație despre programe: în orice moment accesează o **zonă relativ mică** a spațiului de adrese și **reutilizează** datele și instrucțiunile folosite recent.

T Localitate temporală (în timp)

O locație accesată acum va fi **foarte probabil accesată din nou** curând. Cauza: **bucle** și **funcții** apelate repetat.

S Localitate spațială (în spațiu)

Locațiile **vecine** cu una accesată recent vor fi probabil accesate curând. Cauza: instrucțiuni **secvențiale**, tablouri parcurse element cu element.

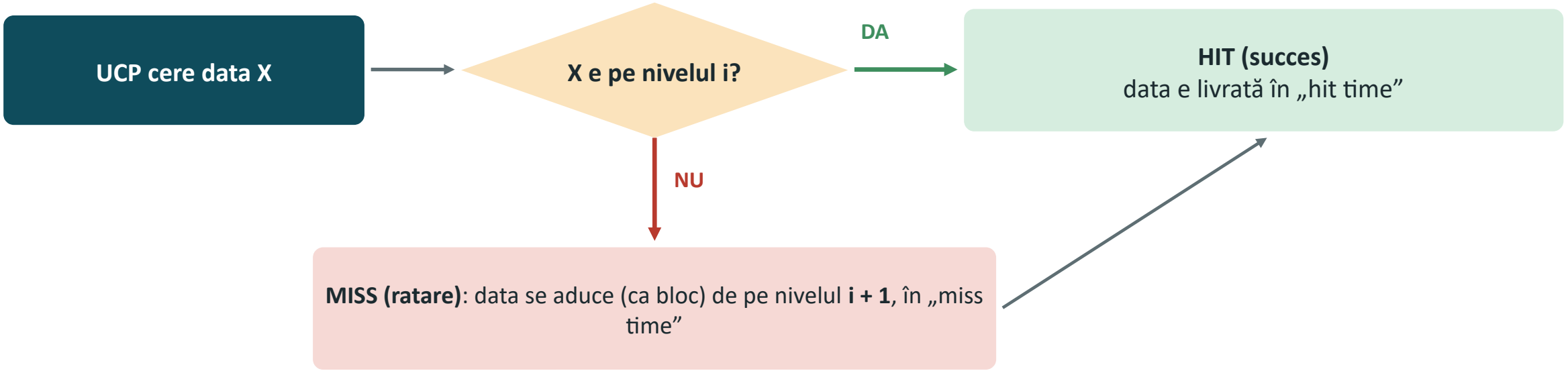
```
int s = 0;
for (int i = 0; i < 100; i++)
    s = s + a[i];
```

exemplu în C (ilustrativ)

1. **s, i** și instrucțiunile buclei sunt folosite de 100 de ori → **temporală**
2. **a[0], a[1], a[2]…** sunt la adrese consecutive → **spațială**
3. Ierarhia păstrează datele recente **aproape de UCP** și aduce **blocuri contigue**

Succes (hit) și ratare (miss)

Cerem o dată pe nivelul **i** al ierarhiei:



Hit time

timpul de a prelua/stoca date **la acel nivel**

Miss time

timpul de a aduce un **bloc** de la nivelurile inferioare

Performanța unui nivel

dată de numărul de **hits** și **misses** și de viteza lor

Exemplu ilustrativ: timpul mediu de acces cu cache

Date (ilustrative, nu sunt în curs): cache cu acces în **2 ns**, memorie principală cu acces în **60 ns**, rata de succes (hit rate) **h = 95%**.

$$t_{mediu} = h \cdot t_{cache} + (1 - h) \cdot (t_{cache} + t_{principală})$$

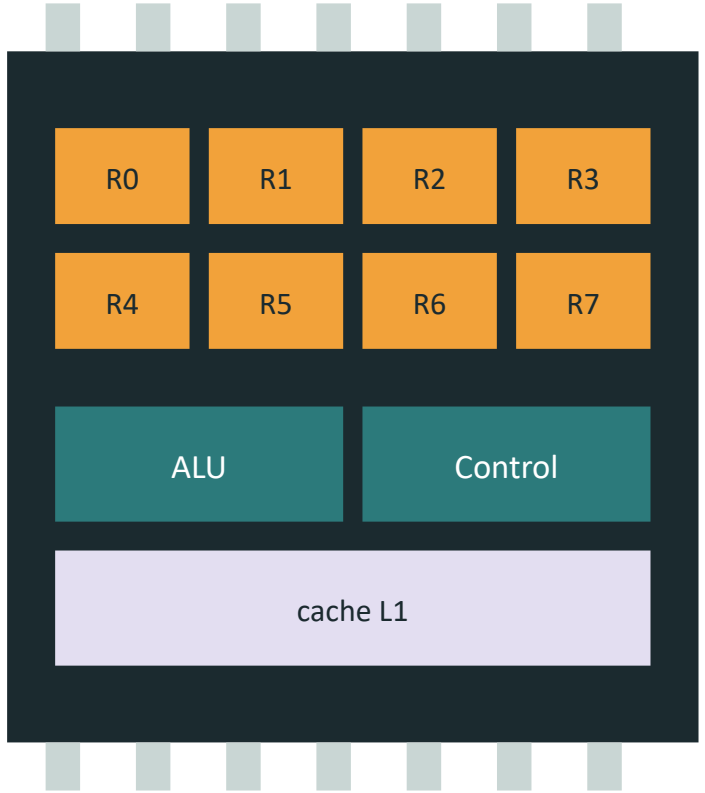
1. Cazul hit (95% din accesări): **2 ns**
2. Cazul miss (5%): se verifică cache-ul, apoi memoria principală: 2 + 60 = **62 ns**
3. $t_{mediu} = 0.95 \cdot 2 + 0.05 \cdot 62 = 1.9 + 3.1 =$ **5 ns**
4. Fără cache, fiecare acces ar dura **60 ns** → de **12 ori** mai lent

Concluzie: datorită localității referințelor, hit rate-ul real este ridicat → memoria pare aproape la fel de rapidă ca cache-ul, dar are capacitatea memoriei principale.

Nivelul de vârf: regiștrii interni ai procesorului

Caracteristici

- Tipul de memorie **cel mai rapid, cel mai mic și cel mai scump**
- Un procesor are de obicei **câteva zeci** de regiștri (unele sisteme: **sute**)
- Stocare **temporară** de date și instrucțiuni
- Formează **memoria locală** a procesorului – nivelul cel mai apropiat de UCP



schemă ilustrativă a unui procesor

Memoria principală: RAM și ROM

Al doilea nivel: principala memorie internă – **rapidă**, separată fizic de procesor, cu **capacitate relativ mare**; fiecare locație are o **adresă unică**. Tehnologie: **circuite integrate semiconductoare**.

RAM – Random Access Memory

- Permite **scriere** (stocarea de informații noi) **și citire**
- Moduri de operare: **static (SRAM)** și **dinamic (DRAM)**
- **Volatilă**

ROM – Read Only Memory

- Permite **numai citire**
- Conținut fixat; nu poate fi modificat de utilizator
- **Nevolatilă**

Volatil vs. nevolatil

Volatilă = pierde informațiile când alimentarea e oprită (RAM static și dinamic – celulele au nevoie de alimentare). **Nevolatilă** = păstrează informațiile fără alimentare (disc magnetic, ROM, flash).

SRAM vs. DRAM

Caracteristică	SRAM (static)	DRAM (dinamic)
Celula de memorie	circuite **flip-flop**	**sarcini electrice** în condensatoare (tranzistoare MOS)
Păstrarea datei	nedefinit, cât timp e alimentată	condensatorul se descarcă → **reîmprospătare** la câteva ms
Viteză	**mai rapidă (cicluri citire/scriere mai scurte)**	mai lentă
Mărimea celulei / densitate	celulă mai mare, mai complexă	**celulă mai simplă și mică → mai densă**
Consum / capacitate pe cip	mai mare / mai mică	**consum redus, capacitate mai mare**
Cost	mai scumpă	**mai ieftină**
Ușurință în utilizare	**mai ușor de utilizat**	necesită circuite de reîmprospătare

Observație: **reîmprospătarea** = citirea datelor din memorie și rescrierea lor la **aceleași adrese**, ciclic. Ambele tipuri sunt **volatile**.

Memoria ROM: BIOS, PROM, EPROM, EEPROM

ROM

- Poate fi doar **citită**; nevolatilă; conținut permanent, nu se transferă de pe alt dispozitiv
- **Inscripționată la fabricație** – cost mare, nu admite erori (scrisă o singură dată)
- Celula: o singură **diodă** sau un **tranzistor** (mai simplă decât celula RAM)

ROM BIOS

- **Basic Input Output System** – interfața între componente și sistemul de operare
- Stochează permanent programul inițial de pornire, care **lansează sistemul de operare**

ROM → **PROM** → **EPROM** → **EEPROM**

scrisă de producător la fabricație

programabilă **o singură dată**, după fabricație

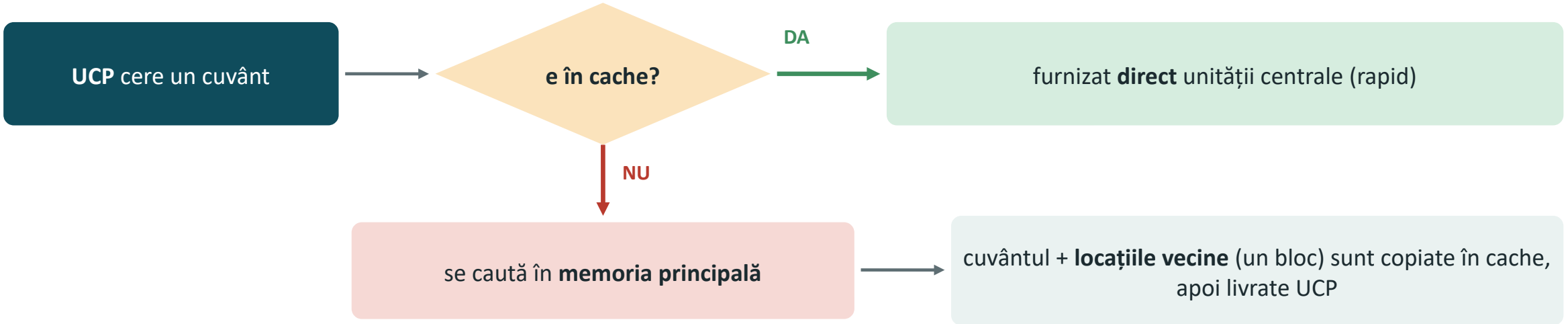
în plus, permite **ștergerea** datelor

ștergere **electrică selectivă**; reprogramabilă de sistem – cea mai avantajoasă

flexibilitate crescătoare →

Memoria cache – cum funcționează

Memorie **intermediară**, mai mică dar mai rapidă decât memoria principală, poziționată între **regiștri** și **memoria principală**. Conține **copii** ale zonelor de program și de date **active** la un moment dat.



Localitate temporală

blocul adus pentru o referință va fi probabil cerut **din nou** curând

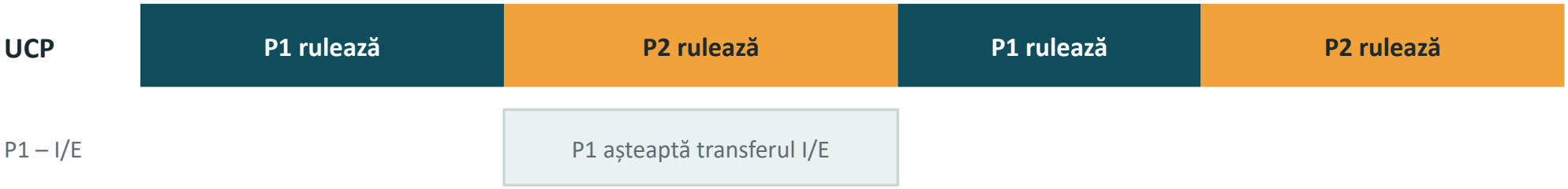
Localitate spațială

se aduc și **vecinii** datei → următoarea regăsire vine direct din cache

*Azi cache-ul este mai mare și organizat pe **mai multe nivele** (L1, L2, L3).*

Multiprogramarea și managementul memoriei

Multiprogramarea: UCP procesează **concomitent** mai multe programe independente; în ierarhia de memorie se află simultan două sau mai multe programe.



UCP nu stă inactivă: cât P1 așteaptă I/E, rulează P2.

De ce e nevoie de gestiune

Memoria principală nu e suficientă: un program stă **în memoria auxiliară** și, la execuție, e adus **în întregime sau parțial** în memoria principală.

Sistemul de management al memoriei

Componenta care **supraveghează fluxul** de informații între memoria auxiliară și memoria principală (vezi secțiunea 5).

Memoria externă (auxiliară)

Nivelul cel mai de jos: dispozitive cu **capacitate mare** dar **viteze de acces mai mici** – discuri și benzi magnetice, discuri optice etc.

mod de acces | **timp de acces** | **rată de transfer** | **capacitate** | **cost**

↑ *cele mai importante caracteristici*

Timpul de acces la benzi/discuri

- **timp de căutare**: poziționarea capului de citire-scriere
- **+ timp de transfer**: mutarea datelor către/de pe dispozitiv
- Căutarea durează **mult mai mult** decât transferul → datele se organizează în **înregistrări / blocuri**

Rata de transfer

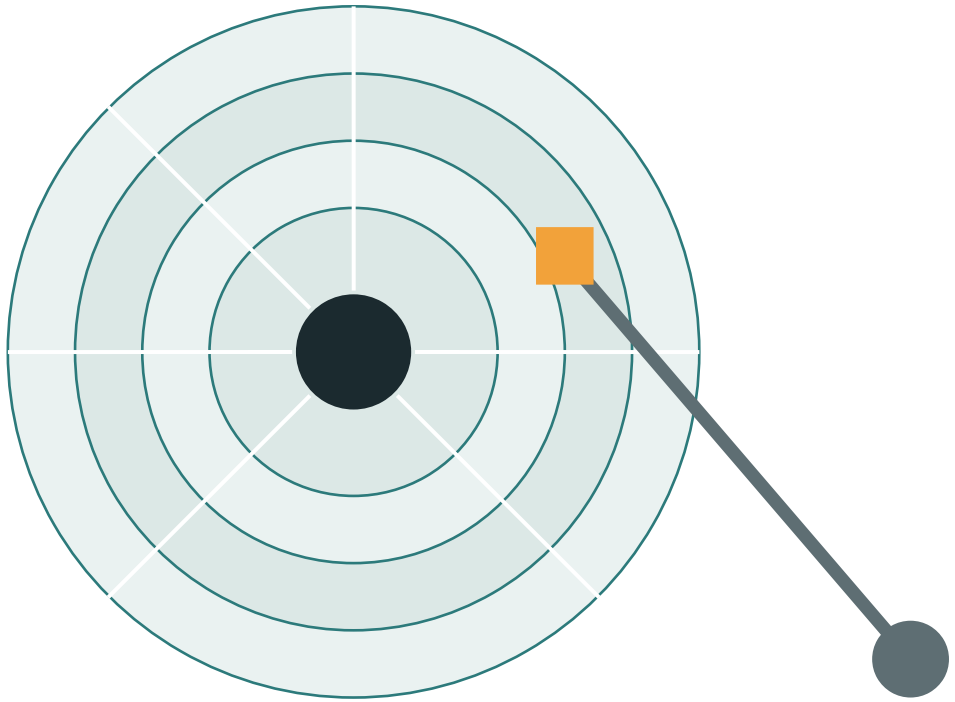
Numărul de **caractere sau cuvinte** transferate pe secundă **după** ce dispozitivul a fost poziționat la începutul înregistrării.

O **înregistrare** = un anumit număr de caractere sau cuvinte.

Hard-disk-ul: structura

platan (disc)

pistă = cerc concentric



sector

cap de citire/scriere pe **braț mobil**

Componente și termeni

- Mai multe **platane** concentrice, magnetizate pe **ambele fețe**, fiecare față cu **propriul cap**
- Rotație constantă: **3600 / 4500 / 5200 / 5400 / 7200** rot/min
- **Piste** împărțite în **sectoare**, separate de **intersector gap** (fără date)
- **Sectorul** = cea mai mică unitate transferabilă (unitatea de adresare)
- Capul **nu atinge** discul – plutește la distanță extrem de mică
- **Placa logică (controlerul)** comandă unitatea și transferul disc ↔ memorie

Capacitatea unui hard-disk

Constantele de construcție: numărul de **fețe active**, de **piste pe față**, de **sectoare pe pistă** și cantitatea de date pe **sector**.

Capacitate = fețe × piste/față × sectoare/pistă × bytes/sector

fețe câte capete	×	**piste/față** cercuri concentrice	×	**sectoare/pistă** felii pe cerc	×	**bytes/sector** de obicei 512 B

Observație: aplicăm formula în **Problema 1** din testul de autoevaluare (rezolvată pas cu pas la final). Dacă fiecare platan are două fețe active, numărul de fețe = 2 × numărul de platane – verificați mereu ce date dă enunțul.

Timpul mediu de acces la disc – componentele



proporții orientative (exemplul 1)

Timp de poziționare (seek time): mutarea brațului pe pista dorită

Întârziere de rotație (latența): așteptarea sectorului sub cap

Timp de transfer: dimensiune sector / rata de transfer

Overhead controller: operarea controller-ului

$$t_{acces} = t_{poziționare} + \text{întârzierea medie} + t_{transfer} + \text{overhead controller}$$

Întârzierea de rotație – pas cu pas

Întârzierea medie = 0.5 / (nr. rotații pe secundă) — în medie sectorul dorit e la **jumătate de rotație** distanță

Viteză	Pas 1: rot/s = rpm / 60	Pas 2: 0.5 / (rot/s)	Rezultat
3600 rpm	3600 / 60 = 60 rot/s	0.5 / 60 = 0.00833 s	**8.33 ms**
5200 rpm	5200 / 60 ≈ 86.67 rot/s	0.5 / 86.67 ≈ 0.00577 s	**≈ 5.77 ms**
7200 rpm	7200 / 60 = 120 rot/s	0.5 / 120 ≈ 0.004167 s	**≈ 4.17 ms**

Observație: cursul scrie 5.76 ms și 4.16 ms (valori trunchiate); rotunjit corect: 5.77 ms și 4.17 ms – diferența e neglijabilă. **Discul mai rapid → latență mai mică.** De regulă: câțiva ms pentru poziționare (sub 10 ms) + câțiva ms de rotație.

Exemplul 1 din curs: timpul mediu de acces – pas cu pas

Parametru	Valoare
dimensiunea sectorului	512 B = 0.5 KB
viteza de rotație	7200 rpm
timp mediu de poziționare	6 ms
rata de transfer	50 MB/s
overhead controller	0.2 ms

1. Întârzierea medie: 7200 rpm = 120 rot/s → 0.5 / 120 ≈ **4.16 ms**
2. Timp de transfer: 0.5 KB / 50 MB/s = 0.5 / 51 200 s ≈ **0.01 ms**
3. Poziționare: **6 ms**; overhead controller: **0.2 ms**
4. Adunăm: 6 + 4.16 + 0.01 + 0.2 = **10.37 ms**



transfer 0.01 + controller 0.2 →

Concluzie: aproape tot timpul se pierde în **mișcări mecanice** (poziționare + rotație); transferul propriu-zis e neglijabil.

Alte medii de stocare externă

F Memoria flash

- Un tip de **EEPROM**
- Memorie **semiconductoare nevolatilă**
- Nu pierde datele la deconectare; **nu are nevoie de alimentare** pentru a păstra informația
- Ex.: stick USB, carduri de memorie, SSD

O Discuri optice

Stocare prin mijloace **optice (laser)** atât la scriere, cât și la citire.

ROM
doar citire

WORM
write-once, read-many

WMRM
rescriptibile: write-many, read-many

Disc	Semnificație
CD-ROM	Compact Disk Read-Only Memory
CD-RW	CD Rewritable
DVD	Digital Versatile Disk
DVD-R / DVD-RW	DVD Recordable / Rewritable
Blu-ray	disc video de înaltă definiție

5

Adresarea memoriei

Managementul memoriei de către sistemul de operare · fragmentare · memorie virtuală · paginare · MMU, tabela de pagini, TLB

Obiectivele managementului memoriei

Memoria e **partajată** de procese tot mai mari și mai numeroase → împărțirea și gestionarea ei este o sarcină esențială a **sistemului de operare**.

1 Alocare eficientă

cât mai multe procese încărcate în limita memoriei fizice

2 Protecție

un proces nu poate citi/scrie memoria altuia fără permisiune

3 Partajare

procesele care colaborează la un task pot accesa o zonă comună de date

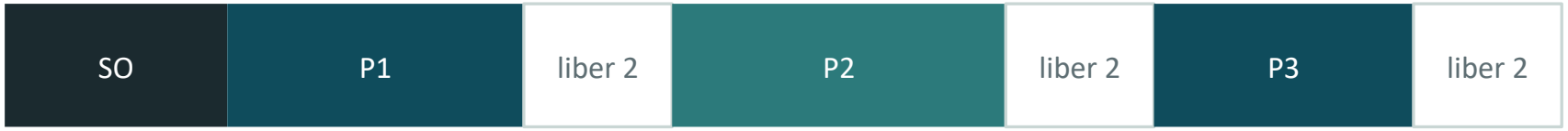
4 Performanță

individuală și de ansamblu – câte procese pot fi susținute simultan

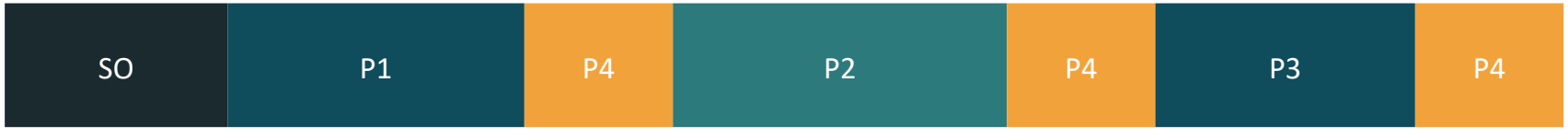
Fragmentarea și alocarea necontiguă

Fragmentare: în memorie rămân zone **libere, dar prea mici** pentru a susține un proces → scad eficiența și performanța.

Situația: procesul P4 are nevoie de **6 unități**; există 6 unități libere în total, dar în 3 bucăți de câte 2.



Alocare contiguă → imposibilă (nicio zonă liberă de 6).



Alocare necontiguă → P4 primește cele 3 zone neadiacente libere ✓

Concluzie: când SO nu găsește o zonă suficient de mare, alocă procesului **mai multe zone neadiacente, libere**. Aceasta e baza **paginării**. (Dimensiunile sunt ilustrative.)

Memoria virtuală

SO păstrează în memorie **doar anumite părți** ale unui proces când acesta depășește memoria. Mecanismul e **automat** și **invizibil** utilizatorului: procesele rulează fără să fie integral în memorie, iar totalul memoriei ocupate **poate depăși** memoria fizică.

Adresă virtuală

Adresa folosită **în program**. Mulțimea lor = **spațiul de adresare** (spațiul de adrese virtuale).

Adresă fizică

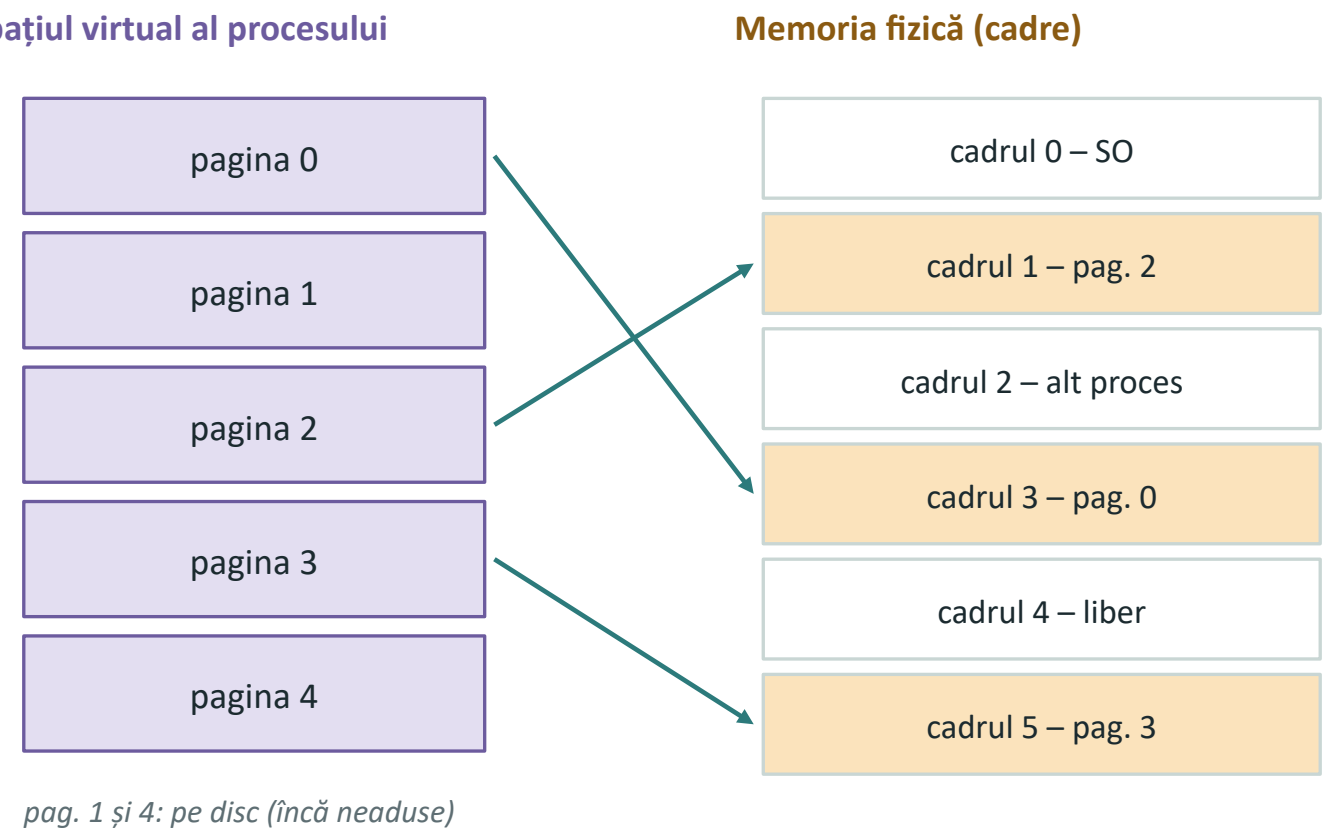
Adresa folosită de **procesor** pentru memoria fizică (o **locație**). Mulțimea lor = **spațiul de memorie** (memoria hardware).



*În calculatoarele cu memorie virtuală: **spațiul de adrese virtuale > spațiul de memorie**.*

Paginarea: pagini și cadre-pagină

Ambele spații se împart în grupuri de **aceeași dimensiune** (de obicei **4 K**): spațiul virtual în **pagini**, memoria fizică în **cadre-pagină** (page frames).



pag. 1 și 4: pe disc (încă neaduse)

Observație: se aduc în memorie doar paginile **necesare imediat**; la nevoie, paginile altor procese se mută **pe disc** ca să facă loc. Cadrele **nu trebuie să fie consecutive**.

Alternativă: segmentarea

spațiul logic împărțit în blocuri de **lungime variabilă** (cu o dimensiune maximă).

MMU, tabela de pagini și TLB

Translatarea adreselor se face **dinamic**, în timpul execuției, automat, printr-un **tabel de translatare**. Eficiența ei influențează performanța memoriei virtuale.

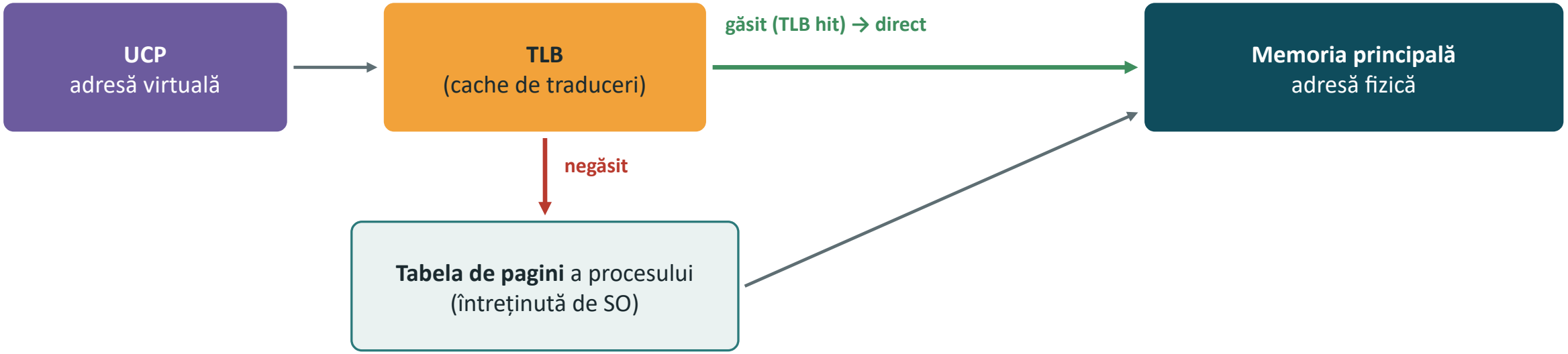


Tabela de pagini

indexată după **numărul paginii virtuale**; fiecare intrare conține **numărul paginii fizice** (dacă pagina e în memorie)

TLB – Translation Look-aside Buffer

cache pentru intrările cele mai recent folosite; parte a **MMU**; mărește viteza

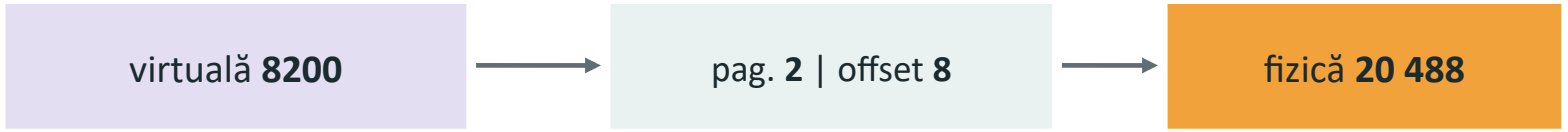
Exemplu ilustrativ: traducerea unei adrese virtuale

Date (ilustrative, nu sunt în curs): pagini de **4 KB = 4096 B**; programul accesează adresa virtuală **8200**; tabela de pagini:

Pagina virtuală	Cadru fizic
0	3
1	pe disc
2	**5**
3	1

Observație: dacă pagina ar fi fost „pe disc” (ex. pagina 1), SO ar fi adus-o întâi în memorie.

1. Numărul paginii: $\lfloor 8200 / 4096 \rfloor = \mathbf{2}$
2. Deplasamentul (offset) în pagină: $8200 - 2 \cdot 4096 = 8200 - 8192 = \mathbf{8}$
3. Căutăm pagina 2 în tabelă (după TLB) → **cadrul 5**
4. Adresa fizică = cadru · 4096 + offset = 5 · 4096 + 8 = **20 488**



?

Test de autoevaluare – rezolvări pas cu pas

1. Capacitatea unui hard-disk 2. Timpul de citire/scriere pentru 1025 bytes 3. Cum se numește viteza de găsire a informației?

Problema 1 – capacitatea totală a discului

Hard-disk cu **5 platane**, **1024 piste/platan**, **512 sectoare/pistă**, **512 bytes/sector**. Care este capacitatea totală?

1. Bytes pe pistă: 512 sectoare × 512 B = 262 144 B = 2^{18} B = **256 KB** (= 0.25 MB)

2. Bytes pe platan: 1024 piste × 256 KB = 262 144 KB = **256 MB** (= 0.25 GB)

3. Total: 5 platane × 256 MB = 1280 MB = **1.25 GB** (1 342 177 280 B)

Atenție: răspunsul din curs folosește **0.2 MB/pistă** și obține **1 GB**, dar 512 × 512 B = 262 144 B = **0.25 MB**, deci rezultatul corect este **1.25 GB** (cu unități binare). Verificare: $5 \cdot 2^{10} \cdot 2^{9} \cdot 2^{9} = 5 \cdot 2^{28}$ B = $1.25 \cdot 2^{30}$ B.

Enunțul dă „piste per platan”, deci nu mai înmulțim cu numărul de fețe.

Problema 2 – timpul de citire/scriere pentru 1025 bytes

Parametru	Valoare
viteza de rotație	7200 rpm
timp mediu de poziționare	5 ms
rata de transfer	50 MB/s
overhead controller	0.2 ms
date de transferat	1025 B ≈ 1 KB

1. Întârzierea medie: 7200 / 60 = 120 rot/s → 0.5 / 120 ≈ **4.16 ms**
2. Timp de transfer: 1 KB / 50 MB/s = 1 / 51 200 s ≈ **0.02 ms**
3. Poziționare **5 ms**, overhead **0.2 ms**
4. Total: 5 + 4.16 + 0.02 + 0.2 = **9.38 ms**



Observație: comparativ cu Exemplul 1 (10.37 ms): poziționarea mai rapidă (5 vs. 6 ms) contează mult; dublarea datelor (0.01 → 0.02 ms) aproape deloc.

Problema 3 și rezumatul unității

3. Viteza cu care un disc găsește informațiile căutate se numește:

a. **timp de acces** ✓ b. timp de poziționare
c. seek time d. overhead

Răspuns (curs): **a) timp de acces** – include poziționarea, rotația, transferul și overhead-ul.

1	**Concepte**	3 puncte de vedere; **6 nivele** ierarhice (Tanenbaum)
2	**Componente**	hardware, software, date, comunicare; **UCP = ALU + UC + regiștri**
3	**Performanță**	f, **CPI**, **MIPS**, T = IC·CPI·t; **Amdahl**: 1/[(1−F)+F/N]
4	**Memorii**	acces **secvențial, direct, aleator, asociativ**; latență, ciclu, rată
5	**Ierarhie**	regiștri → cache → RAM → auxiliară; **localitate**; hit/miss; disc
6	**Adresare**	memorie **virtuală**, **paginare**, MMU, tabelă de pagini, **TLB**

Bibliografie: Dinu, S., Pomazan, C. – Programarea calculatoarelor (Nautica, 2013); Kernighan, B. W., Ritchie, D. M. – Limbajul C (Teora, 2003); Năstac, I. (Printech, 2006); Negrescu, L. (Albastră, 2000); Dinu, S., Vulcu, V. (Nautica, 2019). Tanenbaum, A. S. – Structured Computer Organization.